

SIN6U45L : Calculabilité

Introduction, fonctions et algorithmes

Luigi Santocanale
LIS, Aix-Marseille Université

21 janvier 2026

Plan

Bureaucratie

Historique :
la découverte de la notion d'algorithme

Une plongée dans la théorie des ensembles :
il y a plus de fonctions que d'algorithmes

Calculer et décider

Plan

Bureaucratie

Historique :

la découverte de la notion d'algorithme

Une plongée dans la théorie des ensembles :
il y a plus de fonctions que d'algorithmes

Calculer et décider

Présentation

- Luigi Santocanale
- Courriels pour me contacter :
 - ▶ `luigi.santocanale@lis-lab.fr`
 - ▶ `luigi.santocanale@univ-amu.fr`
- Intervenants dans le cours (TDs) :
 - ▶ Nathan Lhote <`nathan.lhote@univ-amu.fr`> ,
 - ▶ Kevin Perrot <`kevin.perrot@univ-amu.fr`>
- Page web du cours sur AMETICE :
`https://ametice.univ-amu.fr/course/view.php?id=142647`

Organisation du cours et M3C

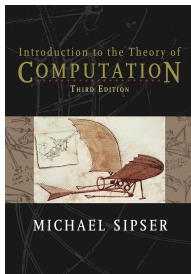
- CM : $1,5h * 4 + 2h * 2$ (10h CMs)
- TD : $7*2h + 4*1,5h$ (20h TDs)
- UE en ECI : Évaluation Continue Intégrale
- Pas de 1^{re} session
- Note finale :

$$NF = \text{Max}(0,25*CC1 + 0,25*CC2 + 0,5*DE ; DE)$$

où DE = épreuve écrite (durée : 1h30).

Bibliographie

- Michael Sipser : Introduction to the Theory of Computation (3rd ed. Cengage Learning).



- Le cours s'appuie sur d'autres ressources disponibles en ligne :
 - ▶ Le cours de Kevin Perrot :
<https://pageperso.lis-lab.fr/kevin.perrot/teaching.html>
 - ▶ Le cours de Pierre Wolper :
<https://people.montefiore.uliege.be/pw/cours/calc.html>

Plan

Bureaucratie

Historique :
la découverte de la notion d'algorithme

Une plongée dans la théorie des ensembles :
il y a plus de fonctions que d'algorithmes

Calculer et décider

La préhistoire

- ~900 : Al-Khwârizmî : ouvrage systématique donnant des solutions aux équations linéaires et quadratiques,
- 1843 : Première implémentation d'un algorithme sous forme de programme (calcul des nombres de Bernoulli par Ada Lovelace).
- ~1800 : Pas de nette distinction entre fonction et algorithme.

La quête pour des algorithmes

David Hilbert, au congrès international des mathématiciens, en 1900 à Paris :

- **De la possibilité de résoudre une équation diophantienne.**¹

« On donne une équation diophantienne à un nombre quelconque d'inconnues et à coefficients entiers rationnels : On demande de trouver

on pourra distinguer si l'équation est résoluble en nombres entiers rationnels. »

- Trouver un algorithme² pour pouvoir répondre à la question suivante :
étant donné un polynôme $p(x_1, \dots, x_n)$, existe-t-il un tuple entier $(\alpha_1, \dots, \alpha_n) \in \mathbb{Z}^n$ tel que $p(\alpha_1, \dots, \alpha_n) = 0$?

1. C'est le 10ème problème de Hilbert

2. Matijasevic 1973 : un tel algorithme n'existe pas ! (Mais il existe bien pour les polynômes à une seule variable.)

La quête pour des algorithmes

David Hilbert, au congrès international des mathématiciens, en 1900 à Paris :

- **De la possibilité de résoudre une équation diophantienne.**¹

« On donne une équation diophantienne à un nombre quelconque d'inconnues et à coefficients entiers rationnels : On demande de trouver

*une méthode par laquelle,
au moyen d'un nombre fini d'opérations,*

on pourra distinguer si l'équation est résoluble en nombres entiers rationnels. »

- Trouver un algorithme² pour pouvoir répondre à la question suivante :

étant donné un polynôme $p(x_1, \dots, x_n)$, existe-t-il un tuple entier $(\alpha_1, \dots, \alpha_n) \in \mathbb{Z}^n$ tel que $p(\alpha_1, \dots, \alpha_n) = 0$?

1. C'est le 10ème problème de Hilbert

2. Matijasevic 1973 : un tel algorithme n'existe pas ! (Mais il existe bien pour les polynômes à une seule variable.)

La quête pour des algorithmes

David Hilbert, au congrès international des mathématiciens, en 1900 à Paris :

- **De la possibilité de résoudre une équation diophantienne.**¹
« On donne une équation diophantienne à un nombre quelconque d'inconnues et à coefficients entiers rationnels : On demande de trouver
un algorithme par lequel

on pourra distinguer si l'équation est résoluble en nombres entiers rationnels. »

- Trouver un algorithme² pour pouvoir répondre à la question suivante :
étant donné un polynôme $p(x_1, \dots, x_n)$, existe-t-il un tuplet entier $(\alpha_1, \dots, \alpha_n) \in \mathbb{Z}^n$ tel que $p(\alpha_1, \dots, \alpha_n) = 0$?

1. C'est le 10ème problème de Hilbert

2. Matijasevic 1973 : un tel algorithme n'existe pas ! (Mais il existe bien pour les polynômes à une seule variable.)

La quête pour des algorithmes

David Hilbert, au congrès international des mathématiciens, en 1900 à Paris :

- **De la possibilité de résoudre une équation diophantienne.**¹
« On donne une équation diophantienne à un nombre quelconque d'inconnues et à coefficients entiers rationnels : On demande de trouver

un algorithme par lequel

on pourra distinguer si l'équation est résoluble en nombres entiers rationnels. »

- Trouver un algorithme² pour pouvoir répondre à la question suivante :
étant donné un polynôme $p(x_1, \dots, x_n)$, existe-t-il un tuple entier $(\alpha_1, \dots, \alpha_n) \in \mathbb{Z}^n$ tel que $p(\alpha_1, \dots, \alpha_n) = 0$?

1. C'est le 10ème problème de Hilbert

2. Matijasevic 1973 : un tel algorithme n'existe pas ! (Mais il existe bien pour les polynômes à une seule variable.)

Le formalisme de Hilbert

- C'est quoi l'infini³ ?
- Tout peut se justifier tant qu'on utilise des méthodes finitaires.
- Méthode finitaire == méthode reposant sur l'arithmétique (élémentaire).
- Peut-on justifier l'arithmétique par des méthodes finitaires ?
- Le 2ème problème de Hilbert @Paris1900 :
Prouver la cohérence de l'arithmétique en utilisant des méthodes finitaires, c'est-à-dire, en utilisant l'arithmétique elle même.
- Le théorème d'incomplétude de Gödel ~1933 :
réponse négative au 2ème problème de Hilbert.
Pour obtenir ce théorème, Gödel développe la théorie des fonctions primitives récurrentes.

Le problème de décision de la logique du 1er ordre

- En 1928, D. Hilbert and W. Ackermann posent cette question :
« y a-t-il un algorithme tel que,
prenant en entrée deux formules γ, δ du premier ordre,
répond vrai ssi

$$\gamma \vdash \delta,$$

c'est-à-dire, δ est conséquence logique de γ ? »

- Réponse négative donnée indépendamment par Alonzo Church, Alan Turing et Emil Post en 1936.
- NB : un tel algorithme existe bien si γ, δ sont des formules de la logique propositionnelle.

Le problème de décision de la logique du 1er ordre

- En 1928, D. Hilbert and W. Ackermann posent cette question :
« y a-t-il un algorithme tel que,
prenant en entrée deux formules γ, δ du premier ordre,
répond vrai ssi

$$\gamma \vdash \delta,$$

c'est-à-dire, δ est conséquence logique de γ ? »

- Réponse négative donnée indépendamment par Alonzo Church, Alan Turing et Emil Post en 1936.
- NB : un tel algorithme existe bien si γ, δ sont des formules de la logique propositionnelle.

Le problème de décision de la logique du 1er ordre

- En 1928, D. Hilbert and W. Ackermann posent cette question :
« y a-t-il un algorithme tel que,
prenant en entrée deux formules γ, δ du premier ordre,
répond vrai ssi

$$\gamma \vdash \delta,$$

c'est-à-dire, δ est conséquence logique de γ ? »

- Réponse négative donnée indépendamment par Alonzo Church, Alan Turing et Emil Post en 1936.
- NB : un tel algorithme existe bien si γ, δ sont des formules de la logique propositionnelle.

1930 : la naissance de la théorie de la calculabilité

- L'œuvre de Gödel donne naissance à la notion de fonction primitive réursive .
- Cette notion fut généralisée par Kleene à celle de fonction μ -réursive .
- Au même temps, on développe indépendamment les notions de fonction λ -calculable (Church), fonction calculée par une *machine de Turing* (Turing), systèmes canoniques (Post) .
- Kleene 1945 démontre l'équivalence entre ces notions : μ -réursive, λ -calculable, Turing calculable, Post canonique. La thèse de Church prend naissance.
- En 1945, Von Neumann propose un des premiers modèles d'ordinateur.

1930 : la naissance de la théorie de la calculabilité

- L'œuvre de Gödel donne naissance à la notion de fonction primitive réursive .
- Cette notion fut généralisée par Kleene à celle de fonction μ -réursive .
- Au même temps, on développe indépendamment les notions de fonction λ -calculable (Church), fonction calculée par une *machine de Turing* (Turing), systèmes canoniques (Post) .
- Kleene 1945 démontre l'équivalence entre ces notions : μ -réursive, λ -calculable, Turing calculable, Post canonique. La thèse de Church prend naissance.
- En 1945, Von Neumann propose un des premiers modèles d'ordinateur.

1930 : la naissance de la théorie de la calculabilité

- L'œuvre de Gödel donne naissance à la notion de fonction primitive réursive .
- Cette notion fut généralisée par Kleene à celle de fonction μ -réursive .
- Au même temps, on développe indépendamment les notions de fonction λ -calculable (Church), fonction calculée par une *machine de Turing* (Turing), systèmes canoniques (Post) .
- Kleene 1945 démontre l'équivalence entre ces notions : μ -réursive, λ -calculable, Turing calculable, Post canonique. La thèse de Church prend naissance.
- En 1945, Von Neumann propose un des premiers modèles d'ordinateur.

Qui est qui ?



Dans ce cours

- Ensembles dénombrables
- Machines de Turing
- Limites de la calculabilité
- Réductibilité des problèmes
- Universalité et thèse de Church

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.
Objectif : connaître

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :
 - Comprendre les notions de calculabilité.
 - Comprendre les limites du calcul, en fait.
- Connaître les origines de l'informatique.
- Comprendre le pourquoi de certaines recherches en informatique :
- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : **s'orienter sur**

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- programmation assembleur,
- théorie de la complexité, en MI,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- ▶ programmation assembleur,
- ▶ théorie de la complexité, en M1,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- ▶ programmation assembleur,
- ▶ théorie de la complexité, en M1,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- ▶ programmation assembleur,
- ▶ théorie de la complexité, en M1,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- ▶ programmation assembleur,
- ▶ théorie de la complexité, en M1,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- calcul quantique,
- limites de la décidabilité.

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- ▶ programmation assembleur,
- ▶ théorie de la complexité, en M1,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- ▶ calcul quantique,
- ▶ limites de la décidabilité.

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- ▶ programmation assembleur,
- ▶ théorie de la complexité, en M1,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- ▶ calcul quantique,
- ▶ limites de la décidabilité.

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :

- ▶ programmation assembleur,
- ▶ théorie de la complexité, en M1,

- Connaître les origines de l'informatique.

- Comprendre le pourquoi de certaines recherches en informatique :

- ▶ calcul quantique,
- ▶ limites de la décidabilité.

- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Étudier la calculabilité ?

- Pas tous les problèmes ont une solution algorithmique.

Objectif : s'orienter sur

quels problèmes $\left(\begin{array}{c} \text{on peut} \\ \text{on ne peut pas} \end{array} \right)$ résoudre.

- Propédeutique :
 - ▶ programmation assembleur,
 - ▶ théorie de la complexité, en M1,
- Connaître les origines de l'informatique.
- Comprendre le pourquoi de certaines recherches en informatique :
 - ▶ calcul quantique,
 - ▶ limites de la décidabilité.
- C'est amusant ! Découvrez dès maintenant les machines de Turing par ici :

<https://turingmachine.io/>
<https://turingmachinesimulator.com/>

Plan

Bureaucratie

Historique :

la découverte de la notion d'algorithme

Une plongée dans la théorie des ensembles :
il y a plus de fonctions que d'algorithmes

Calculer et décider

Ensembles équipotents et subpotents

La théorie des ensembles de Cantor est une *théorie de l'infini* : pas tous les ensemble infinis ont la même taille.

Soient A et B deux ensembles.

Définition. On dit que A et B sont **équipotents** (ou qu'ils ont **même cardinalité**⁴) s'il existe une bijection de A vers B . On écrit alors $|A| = |B|$.

Définition. A est **subpotent** à B (on écrit $|A| \leq |B|$) s'il existe une fonction injective de A vers B .

Théorème. (Cantor-Schröder-Bernstein). A et B sont équipotents si et seulement si A est subpotent à B et B est subpotent à A .

On a donc $|A| = |B|$ ssi $|A| \leq |B|$ et $|B| \leq |A|$.

Car $|A|$ et $|B|$ ne sont pas des nombres, ce résultat est non-trivial.

4. Les notions de cardinal et de cardinalité sont assez difficiles à définir en théorie des ensembles : une classe d'équivalence dont les objets sont des ensembles peut ne pas être un ensemble.

Ensembles équipotents et subpotents

La théorie des ensembles de Cantor est une *théorie de l'infini* : pas tous les ensemble infinis ont la même taille.

Soient A et B deux ensembles.

Définition. On dit que A et B sont **équipotents** (ou qu'ils ont **même cardinalité**⁴) s'il existe une bijection de A vers B . On écrit alors $|A| = |B|$.

Définition. A est **subpotent** à B (on écrit $|A| \leq |B|$) s'il existe une fonction injective de A vers B .

Théorème. (Cantor-Schröder-Bernstein). A et B sont équipotents si et seulement si A est subpotent à B et B est subpotent à A .

On a donc $|A| = |B|$ ssi $|A| \leq |B|$ et $|B| \leq |A|$.

Car $|A|$ et $|B|$ ne sont pas des nombres, ce résultat est non-trivial.

4. Les notions de cardinal et de cardinalité sont assez difficiles à définir en théorie des ensembles : une classe d'équivalence dont les objets sont des ensembles peut ne pas être un ensemble.

Ensembles équipotents et subpotents

La théorie des ensembles de Cantor est une *théorie de l'infini* : pas tous les ensemble infinis ont la même taille.

Soient A et B deux ensembles.

Définition. On dit que A et B sont **équipotents** (ou qu'ils ont **même cardinalité**⁴) s'il existe une bijection de A vers B . On écrit alors $|A| = |B|$.

Définition. A est **subpotent** à B (on écrit $|A| \leq |B|$) s'il existe une fonction injective de A vers B .

Théorème. (Cantor-Schröder-Bernstein). A et B sont équipotents si et seulement si A est subpotent à B et B est subpotent à A .

On a donc $|A| = |B|$ ssi $|A| \leq |B|$ et $|B| \leq |A|$.

Car $|A|$ et $|B|$ ne sont pas des nombres, ce résultat est non-trivial.

4. Les notions de cardinal et de cardinalité sont assez difficiles à définir en théorie des ensembles : une classe d'équivalence dont les objets sont des ensembles peut ne pas être un ensemble.

Ensembles équipotents et subpotents

La théorie des ensembles de Cantor est une *théorie de l'infini* : pas tous les ensemble infinis ont la même taille.

Soient A et B deux ensembles.

Définition. On dit que A et B sont **équipotents** (ou qu'ils ont **même cardinalité**⁴) s'il existe une bijection de A vers B . On écrit alors $|A| = |B|$.

Définition. A est **subpotent** à B (on écrit $|A| \leq |B|$) s'il existe une fonction injective de A vers B .

Théorème. (Cantor-Schröder-Bernstein). A et B sont équipotents si et seulement si A est subpotent à B et B est subpotent à A .

On a donc $|A| = |B|$ ssi $|A| \leq |B|$ et $|B| \leq |A|$.

Car $|A|$ et $|B|$ ne sont pas des nombres, ce résultat est non-trivial.

4. Les notions de cardinal et de cardinalité sont assez difficiles à définir en théorie des ensembles : une classe d'équivalence dont les objets sont des ensembles peut ne pas être un ensemble.

Ensembles équipotents et subpotents

La théorie des ensembles de Cantor est une *théorie de l'infini* : pas tous les ensemble infinis ont la même taille.

Soient A et B deux ensembles.

Définition. On dit que A et B sont **équipotents** (ou qu'ils ont **même cardinalité**⁴) s'il existe une bijection de A vers B . On écrit alors $|A| = |B|$.

Définition. A est **subpotent** à B (on écrit $|A| \leq |B|$) s'il existe une fonction injective de A vers B .

Théorème. (Cantor-Schröder-Bernstein). A et B sont équipotents si et seulement si A est subpotent à B et B est subpotent à A .

On a donc $|A| = |B|$ ssi $|A| \leq |B|$ et $|B| \leq |A|$.

Car $|A|$ et $|B|$ ne sont pas des nombres, ce résultat est non-trivial.

4. Les notions de cardinal et de cardinalité sont assez difficiles à définir en théorie des ensembles : une classe d'équivalence dont les objets sont des ensembles peut ne pas être un ensemble.

Ensembles équipotents et subpotents

La théorie des ensembles de Cantor est une *théorie de l'infini* : pas tous les ensemble infinis ont la même taille.

Soient A et B deux ensembles.

Définition. On dit que A et B sont **équipotents** (ou qu'ils ont **même cardinalité**⁴) s'il existe une bijection de A vers B . On écrit alors $|A| = |B|$.

Définition. A est **subpotent** à B (on écrit $|A| \leq |B|$) s'il existe une fonction injective de A vers B .

Théorème. (Cantor-Schröder-Bernstein). A et B sont équipotents si et seulement si A est subpotent à B et B est subpotent à A .

On a donc $|A| = |B|$ ssi $|A| \leq |B|$ et $|B| \leq |A|$.

Car $|A|$ et $|B|$ ne sont pas des nombres, ce résultat est non-trivial.

4. Les notions de cardinal et de cardinalité sont assez difficiles à définir en théorie des ensembles : une classe d'équivalence dont les objets sont des ensembles peut ne pas être un ensemble.

Des propositions de la théorie des ensembles

Proposition. Soient A, B deux ensembles tels que $A \neq \emptyset$.
Il existe une fonction injective $A \hookrightarrow B$ (c'est-à-dire, $|A| \leq |B|$) si, et seulement si, il existe une fonction surjective $B \twoheadrightarrow A$.

Remarque. La condition suffisante (*s'il existe une fonction surjective $B \twoheadrightarrow A$, alors il existe une fonction injective $A \hookrightarrow B$*) est équivalente de l'axiome du choix.

Proposition. Si $|A| \not\leq |B|$, alors $|B| \leq |A|$.

Des propositions de la théorie des ensembles

Proposition. Soient A, B deux ensembles tels que $A \neq \emptyset$.
Il existe une fonction injective $A \hookrightarrow B$ (c'est-à-dire, $|A| \leq |B|$) si, et seulement si, il existe une fonction surjective $B \twoheadrightarrow A$.

Remarque. La condition suffisante (*s'il existe une fonction surjective $B \twoheadrightarrow A$, alors il existe une fonction injective $A \hookrightarrow B$*) est équivalente de l'axiome du choix.

Proposition. Si $|A| \not\leq |B|$, alors $|B| \leq |A|$.

Ensembles dénombrables

Définition. Un ensemble A est **fini** s'il existe un entier positif n tel que $|A| = |\{1, 2, \dots, n\}|$.

Définition. Un ensemble A est **dénombrable** s'il existe une fonction surjective $\mathbb{N} \rightarrow A$.

Donc, A est dénombrable si $A \neq \emptyset$ et $|A| \leq |\mathbb{N}|$.

Lemme. Soit A un sous-ensemble infini de $\mathbb{N}$. Alors $|A| = |\mathbb{N}|$.

On peut ainsi déduire que $|\mathbb{N}|$ est le plus petit cardinal infini.
Que peut-on dire sur $|\mathbb{R}|$, la taille de l'ensemble des réels ?

Ensembles dénombrables

Définition. Un ensemble A est **fini** s'il existe un entier positif n tel que $|A| = |\{1, 2, \dots, n\}|$.

Définition. Un ensemble A est **dénombrable** s'il existe une fonction surjective $\mathbb{N} \rightarrow A$.

Donc, A est dénombrable si $A \neq \emptyset$ et $|A| \leq |\mathbb{N}|$.

Lemme. Soit A un sous-ensemble infini de $\mathbb{N}$. Alors $|A| = |\mathbb{N}|$.

On peut ainsi déduire que $|\mathbb{N}|$ est le plus petit cardinal infini.
Que peut-on dire sur $|\mathbb{R}|$, la taille de l'ensemble des réels ?

Ensembles dénombrables

Définition. Un ensemble A est **fini** s'il existe un entier positif n tel que $|A| = |\{1, 2, \dots, n\}|$.

Définition. Un ensemble A est **dénombrable** s'il existe une fonction surjective $\mathbb{N} \rightarrow A$.

Donc, A est dénombrable si $A \neq \emptyset$ et $|A| \leq |\mathbb{N}|$.

Lemme. Soit A un sous-ensemble infini de $\mathbb{N}$. Alors $|A| = |\mathbb{N}|$.

On peut ainsi déduire que $|\mathbb{N}|$ est le plus petit cardinal infini.
Que peut-on dire sur $|\mathbb{R}|$, la taille de l'ensemble des réels ?

Ensembles dénombrables

Définition. Un ensemble A est **fini** s'il existe un entier positif n tel que $|A| = |\{1, 2, \dots, n\}|$.

Définition. Un ensemble A est **dénombrable** s'il existe une fonction surjective $\mathbb{N} \rightarrow A$.

Donc, A est dénombrable si $A \neq \emptyset$ et $|A| \leq |\mathbb{N}|$.

Lemme. Soit A un sous-ensemble infini de $\mathbb{N}$. Alors $|A| = |\mathbb{N}|$.

On peut ainsi déduire que $|\mathbb{N}|$ est le plus petit cardinal infini.
Que peut-on dire sur $|\mathbb{R}|$, la taille de l'ensemble des réels ?

Ensembles dénombrables

Définition. Un ensemble A est **fini** s'il existe un entier positif n tel que $|A| = |\{1, 2, \dots, n\}|$.

Définition. Un ensemble A est **dénombrable** s'il existe une fonction surjective $\mathbb{N} \rightarrow A$.

Donc, A est dénombrable si $A \neq \emptyset$ et $|A| \leq |\mathbb{N}|$.

Lemme. Soit A un sous-ensemble infini de $\mathbb{N}$. Alors $|A| = |\mathbb{N}|$.

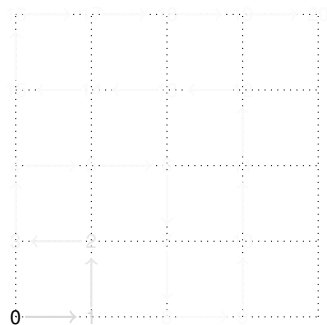
On peut ainsi déduire que $|\mathbb{N}|$ est le plus petit cardinal infini.
Que peut-on dire sur $|\mathbb{R}|$, la taille de l'ensemble des réels ?

Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

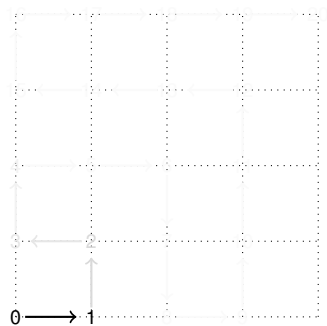


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

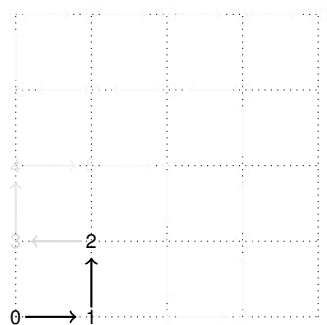


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

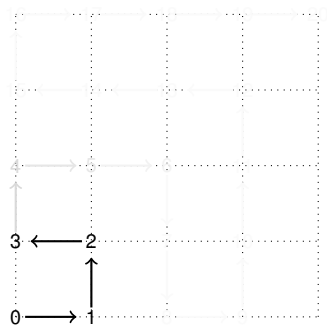


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

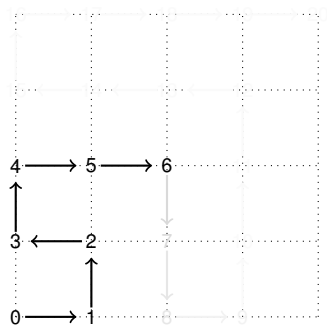


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

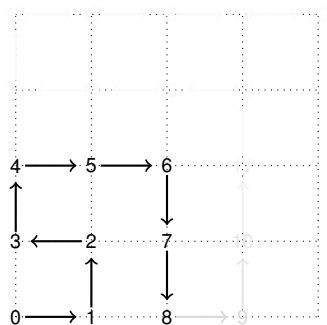


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

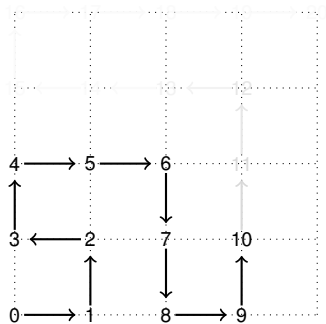


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

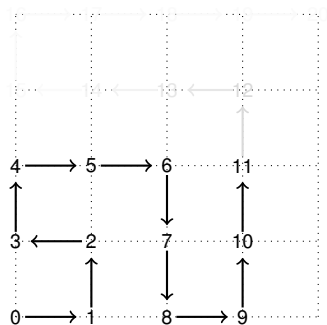


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

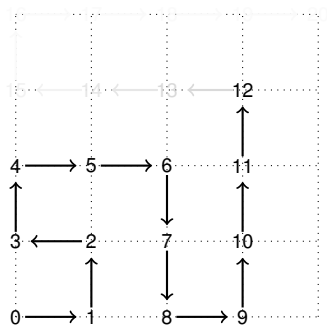


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

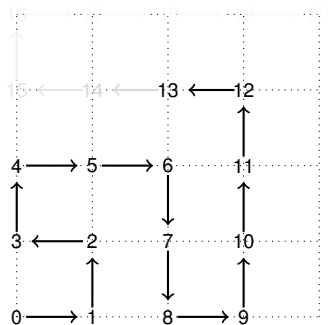


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

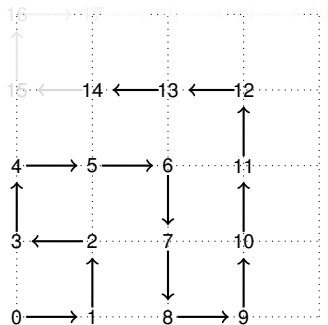


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

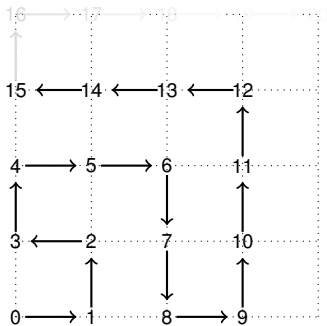


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

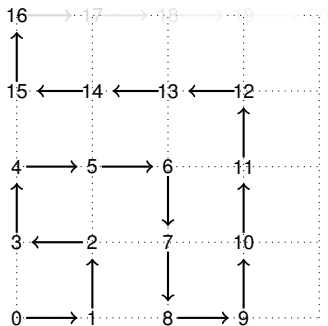


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

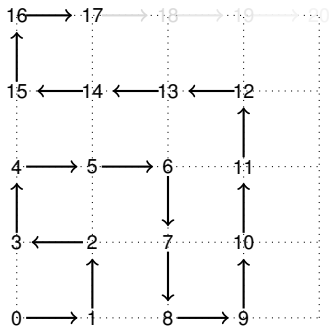


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :

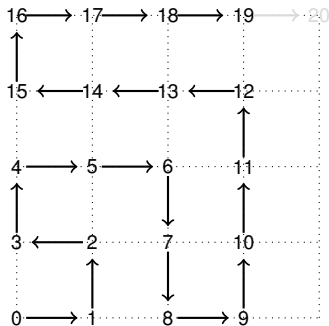


Cardinalité du produit

Lemme. $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

Démonstration.

On « dénombre » les points du plan des entiers (à coordonnées positives ou nulles) :



Cardinalité du monoïde libre

Proposition. Pour un alphabet (ensemble fini de symboles) Σ , on a $|\Sigma^*| = |\mathbb{N}|$.
C'est un exercice de TD.

Corollaire. Pour tout langage de programmation $\mathcal{L}$, l'ensemble de $\Pi^{\mathcal{L}}$ des programmes syntaxiquement corrects est dénombrable ($|\Pi^{\mathcal{L}}| \leq |\mathbb{N}|$).

Démonstration. On a $\Pi^{\mathcal{L}} \subseteq \text{ASCII}^*$, donc

$$|\Pi^{\mathcal{L}}| \leq |\text{ASCII}^*| = |\mathbb{N}|.$$

Car $\Pi^{\mathcal{L}}$ est infini, alors $|\Pi^{\mathcal{L}}| = |\mathbb{N}|$.

Cardinalité de l'ensemble des endofonctions

Lemme. Soit $\mathbb{N}^{\mathbb{N}}$ l'ensemble des fonctions de $\mathbb{N}$ vers $\mathbb{N}$. On a :

$$|\mathbb{N}| < |\mathbb{N}^{\mathbb{N}}|.$$

Démonstration.

$|\mathbb{N}| \leq |\mathbb{N}^{\mathbb{N}}|$: il y a les fonctions constantes.

$|\mathbb{N}^{\mathbb{N}}| \not\leq |\mathbb{N}|$. Par l'absurde : supposons que $\mathbb{N}^{\mathbb{N}}$ soit dénombrable ...



Cardinalité de l'ensemble des endofonctions

Lemme. Soit $\mathbb{N}^{\mathbb{N}}$ l'ensemble des fonctions de $\mathbb{N}$ vers $\mathbb{N}$. On a :

$$|\mathbb{N}| < |\mathbb{N}^{\mathbb{N}}|.$$

Démonstration.

$|\mathbb{N}| \leq |\mathbb{N}^{\mathbb{N}}|$: il y a les fonctions constantes.

$|\mathbb{N}^{\mathbb{N}}| \not\leq |\mathbb{N}|$. Par l'absurde : supposons que $\mathbb{N}^{\mathbb{N}}$ soit dénombrable ...



Cardinalité de l'ensemble des endofonctions

Lemme. Soit $\mathbb{N}^{\mathbb{N}}$ l'ensemble des fonctions de $\mathbb{N}$ vers $\mathbb{N}$. On a :

$$|\mathbb{N}| < |\mathbb{N}^{\mathbb{N}}|.$$

Démonstration.

$|\mathbb{N}| \leq |\mathbb{N}^{\mathbb{N}}|$: il y a les fonctions constantes.

$|\mathbb{N}^{\mathbb{N}}| \not\leq |\mathbb{N}|$. Par l'absurde : supposons que $\mathbb{N}^{\mathbb{N}}$ soit dénombrable ...



Une question avant la preuve

Supposons qu'on puisse énumérer toutes les fonctions $\mathbb{N} \rightarrow \mathbb{N}$:

$$f_0, f_1, f_2, f_3, \dots$$

Soit g définie par :

$$g(i) = f_i(i) + 1$$

Question :

Est-ce que g apparaît dans la liste ?

Réfléchissez 30 secondes...

Une question avant la preuve

Supposons qu'on puisse énumérer toutes les fonctions $\mathbb{N} \rightarrow \mathbb{N}$:

$$f_0, f_1, f_2, f_3, \dots$$

Soit g définie par :

$$g(i) = f_i(i) + 1$$

Question :

Est-ce que g apparaît dans la liste ?

Réfléchissez 30 secondes...

Une question avant la preuve

Supposons qu'on puisse énumérer toutes les fonctions $\mathbb{N} \rightarrow \mathbb{N}$:

$$f_0, f_1, f_2, f_3, \dots$$

Soit g définie par :

$$g(i) = f_i(i) + 1$$

Question :

Est-ce que g apparaît dans la liste ?

Réfléchissez 30 secondes...

Une question avant la preuve

Supposons qu'on puisse énumérer toutes les fonctions $\mathbb{N} \rightarrow \mathbb{N}$:

$$f_0, f_1, f_2, f_3, \dots$$

Soit g définie par :

$$g(i) = f_i(i) + 1$$

Question :

Est-ce que g apparaît dans la liste ?

Réfléchissez 30 secondes...

Méthode diagonale

	0	1	2	3	4	...
f_0	0	1	2	3	5	
f_1	56	21	10	0	1	
f_2	2	5	7	11	13	
f_3	2	4	6	8	10	
f_4	3	6	9	12	15	
$\vdots$						

Méthode diagonale

	0	1	2	3	4	...
f_0	0	1	2	3	5	
f_1	56	21	10	0	1	
f_2	2	5	7	11	13	
f_3	2	4	6	8	10	
f_4	3	6	9	12	15	
$\vdots$						

Méthode diagonale

	0	1	2	3	4	...
f_0	1	1	2	3	5	
f_1	56	22	10	0	1	
f_2	2	5	8	11	13	
f_3	2	4	6	9	10	
f_4	3	6	9	12	16	
$\vdots$						

Il y a des fonctions qui ne sont pas calculables

Une fonction

$$f : \mathbb{N} \rightarrow \mathbb{N}$$

est **calculable/programmable** s'il existe un programme $\pi \in \Pi^{\mathcal{L}}$ tel que, pour tout $n \in \mathbb{N}$, $\pi(n) = f(n)$. C'est-à-dire, π , avec en paramètre n , retourne $f(n)$.

Corollaire. Ils existent des fonctions de $\mathbb{N}^{\mathbb{N}}$ non-programmables.

Démonstration. Si toute fonction de $\mathbb{N}^{\mathbb{N}}$ est programmable, alors il existe un sous-ensemble $\mathcal{F}^5$ de $\Pi^{\mathcal{L}}$ et une fonction surjective de $\mathcal{F}$ vers $\mathbb{N}^{\mathbb{N}}$.

On a alors

$$|\mathbb{N}^{\mathbb{N}}| \leq |\mathcal{F}| \leq |\Pi^{\mathcal{L}}| = |\mathbb{N}|.$$

Mais on a vu que $|\mathbb{N}| < |\mathbb{N}^{\mathbb{N}}|$, ce qui est équivalent à $\mathbb{N}^{\mathbb{N}} \not\leq |\mathbb{N}|$.

Contradiction.

5. C'est l'ensemble des programmes de $\mathcal{L}$ dont le type des entrées et des sorties est $\mathbb{N}$.

Il y a des fonctions qui ne sont pas calculables

Une fonction

$$f : \mathbb{N} \rightarrow \mathbb{N}$$

est **calculable/programmable** s'il existe un programme $\pi \in \Pi^{\mathcal{L}}$ tel que, pour tout $n \in \mathbb{N}$, $\pi(n) = f(n)$. C'est-à-dire, π , avec en paramètre n , retourne $f(n)$.

Corollaire. Ils existent des fonctions de $\mathbb{N}^{\mathbb{N}}$ non-programmables.

Démonstration. Si toute fonction de $\mathbb{N}^{\mathbb{N}}$ est programmable, alors il existe un sous-ensemble $\mathcal{F}^5$ de $\Pi^{\mathcal{L}}$ et une fonction surjective de $\mathcal{F}$ vers $\mathbb{N}^{\mathbb{N}}$.

On a alors

$$|\mathbb{N}^{\mathbb{N}}| \leq |\mathcal{F}| \leq |\Pi^{\mathcal{L}}| = |\mathbb{N}|.$$

Mais on a vu que $|\mathbb{N}| < |\mathbb{N}^{\mathbb{N}}|$, ce qui est équivalent à $\mathbb{N}^{\mathbb{N}} \not\leq |\mathbb{N}|$.

Contradiction.

5. C'est l'ensemble des programmes de $\mathcal{L}$ dont le type des entrées et des sorties est $\mathbb{N}$.

Récapitulation : ce qu'on a démontré

- $|\mathbb{N}|$ est le plus petit cardinal infini.

- $|\mathbb{N} \times \mathbb{N}| = |\Sigma^*| = |\Pi(\mathcal{L})| = |\mathbb{N}|$.

En particulier, l'ensemble des programmes est dénombrable.

- $|\mathbb{N}^{\mathbb{N}}| > |\mathbb{N}|$ (argument diagonal de Cantor).

- L'ensemble des programmes est dénombrable.
 - Il existe donc une suite $p_0 \rightarrow p_1 \rightarrow \dots$ qui énumère tous les programmes.
 - La suite $p_0, p_1, p_2, \dots$ est-elle une suite de programmes qui s'arrête ?

Récapitulation : ce qu'on a démontré

- $|\mathbb{N}|$ est le plus petit cardinal infini.

- $|\mathbb{N} \times \mathbb{N}| = |\Sigma^*| = |\Pi(\mathcal{L})| = |\mathbb{N}|$.

En particulier, l'ensemble des programmes est dénombrable.

- $|\mathbb{N}^{\mathbb{N}}| > |\mathbb{N}|$ (argument diagonal de Cantor).

- Théorème de Gödel : il existe des énoncés φ tels que $\vdash \varphi \leftrightarrow \neg \text{Prov}(\ulcorner \varphi \urcorner)$.
Il existe donc des énoncés φ tels que φ n'est ni prouvable ni réfutable.
Les énoncés φ tels que $\vdash \varphi$ sont donc strictement inférieurs à $|\mathbb{N}^{\mathbb{N}}|$.

Récapitulation : ce qu'on a démontré

- $|\mathbb{N}|$ est le plus petit cardinal infini.
- $|\mathbb{N} \times \mathbb{N}| = |\Sigma^*| = |\Pi(\mathcal{L})| = |\mathbb{N}|$.
En particulier, l'ensemble des programmes est dénombrable.
- $|\mathbb{N}^{\mathbb{N}}| > |\mathbb{N}|$ (argument diagonal de Cantor).

Conclusion

Il existe des fonctions $\mathbb{N} \rightarrow \mathbb{N}$ qui ne sont pas programmables.

La classe des fonctions récursives est strictement incluse dans

Récapitulation : ce qu'on a démontré

- $|\mathbb{N}|$ est le plus petit cardinal infini.
- $|\mathbb{N} \times \mathbb{N}| = |\Sigma^*| = |\Pi(\mathcal{L})| = |\mathbb{N}|$.
En particulier, l'ensemble des programmes est dénombrable.
- $|\mathbb{N}^{\mathbb{N}}| > |\mathbb{N}|$ (argument diagonal de Cantor).

Conclusion :

Il existe des fonctions $\mathbb{N} \rightarrow \mathbb{N}$ qui ne sont pas programmables.

La plupart des fonctions ne sont pas calculables !

Récapitulation : ce qu'on a démontré

- $|\mathbb{N}|$ est le plus petit cardinal infini.
- $|\mathbb{N} \times \mathbb{N}| = |\Sigma^*| = |\Pi(\mathcal{L})| = |\mathbb{N}|$.
En particulier, l'ensemble des programmes est dénombrable.
- $|\mathbb{N}^{\mathbb{N}}| > |\mathbb{N}|$ (argument diagonal de Cantor).

- Conclusion :

Il existe des fonctions $\mathbb{N} \rightarrow \mathbb{N}$ qui ne sont pas programmables.

La plupart des fonctions ne sont pas calculables !

Récapitulation : ce qu'on a démontré

- $|\mathbb{N}|$ est le plus petit cardinal infini.
- $|\mathbb{N} \times \mathbb{N}| = |\Sigma^*| = |\Pi(\mathcal{L})| = |\mathbb{N}|$.
En particulier, l'ensemble des programmes est dénombrable.
- $|\mathbb{N}^{\mathbb{N}}| > |\mathbb{N}|$ (argument diagonal de Cantor).

- **Conclusion :**

Il existe des fonctions $\mathbb{N} \rightarrow \mathbb{N}$ qui ne sont pas programmables.

La plupart des fonctions ne sont pas calculables !

Récapitulation : ce qu'on a démontré

- $|\mathbb{N}|$ est le plus petit cardinal infini.
- $|\mathbb{N} \times \mathbb{N}| = |\Sigma^*| = |\Pi(\mathcal{L})| = |\mathbb{N}|$.
En particulier, l'ensemble des programmes est dénombrable.
- $|\mathbb{N}^{\mathbb{N}}| > |\mathbb{N}|$ (argument diagonal de Cantor).

- **Conclusion :**

Il existe des fonctions $\mathbb{N} \rightarrow \mathbb{N}$ qui ne sont pas programmables.

La plupart des fonctions ne sont pas calculables !

Plan

Bureaucratie

Historique :

la découverte de la notion d'algorithme

Une plongée dans la théorie des ensembles :
il y a plus de fonctions que d'algorithmes

Calculer et décider

La question centrale

On sait maintenant qu'il existe des fonctions non-programmables.

Mais quelles fonctions *sont* calculables ?

Pour répondre à cette question, il faut d'abord **formaliser** la notion de calculabilité.

On commence par des définitions informelles.

La question centrale

On sait maintenant qu'il existe des fonctions non-programmables.

Mais quelles fonctions *sont* calculables ?

Pour répondre à cette question, il faut d'abord **formaliser** la notion de calculabilité.

On commence par des définitions informelles.

La question centrale

On sait maintenant qu'il existe des fonctions non-programmables.

Mais quelles fonctions *sont* calculables ?

Pour répondre à cette question, il faut d'abord **formaliser** la notion de calculabilité.

On commence par des définitions informelles.

La question centrale

On sait maintenant qu'il existe des fonctions non-programmables.

Mais quelles fonctions *sont* calculables ?

Pour répondre à cette question, il faut d'abord **formaliser** la notion de calculabilité.

On commence par des définitions informelles.

Calculer (une définition informelle)

Une fonction

$$f : A \rightarrow B$$

est **calculable** s'il existe un algorithme qui la calcule.

NB : notre compréhension de ce que c'est un algorithme est encore intuitive.

Décider (déf. informelle)

Soit A un sous-ensemble de B . La **fonction caractéristique** de A est la fonction

$$\chi_A : B \rightarrow \{0, 1\},$$

définie par :

$$\chi_A(b) = \begin{cases} 1, & \text{si } b \in A, \\ 0, & \text{sinon.} \end{cases}$$

Définition. Un sous-ensemble $A \subseteq B$ est **décidable** si (et seulement si) la fonction $\chi_A : B \rightarrow \{0, 1\}$ est calculable—c'est-à-dire, s'il existe un algorithme qui calcule χ_A .

Décider (déf. informelle)

Soit A un sous-ensemble de B . La **fonction caractéristique** de A est la fonction

$$\chi_A : B \rightarrow \{0, 1\},$$

définie par :

$$\chi_A(b) = \begin{cases} 1, & \text{si } b \in A, \\ 0, & \text{sinon.} \end{cases}$$

Définition. Un sous-ensemble $A \subseteq B$ est **décidable** si (et seulement si) la fonction $\chi_A : B \rightarrow \{0, 1\}$ est calculable—c'est-à-dire, s'il existe un algorithme qui calcule χ_A .

Calculer revient à décider

Soit $f : A \rightarrow B$ une fonction.

Le graphe de f est le sous-ensemble

$$G_f := \{(x, f(x)) \mid x \in A\} \subseteq A \times B.$$

Évidemment :

$f : A \rightarrow B$ est calculable ssi $G_f \subseteq A \times B$ est décidable.

On a déjà remarqué (par définition) :

$A \subseteq B$ est décidable ssi $\chi_A : B \rightarrow \{0, 1\}$ est calculable.

Calculer revient à décider

Soit $f : A \rightarrow B$ une fonction.

Le graphe de f est le sous-ensemble

$$G_f := \{(x, f(x)) \mid x \in A\} \subseteq A \times B.$$

Évidemment :

$f : A \rightarrow B$ est calculable ssi $G_f \subseteq A \times B$ est décidable.

On a déjà remarqué (par définition) :

$A \subseteq B$ est décidable ssi $\chi_A : B \rightarrow \{0, 1\}$ est calculable.

Calculer revient à décider

Soit $f : A \rightarrow B$ une fonction.

Le graphe de f est le sous-ensemble

$$G_f := \{ (x, f(x)) \mid x \in A \} \subseteq A \times B.$$

Évidemment :

$f : A \rightarrow B$ est calculable ssi $G_f \subseteq A \times B$ est décidable.

On a déjà remarqué (par définition) :

$A \subseteq B$ est décidable ssi $\chi_A : B \rightarrow \{0, 1\}$ est calculable.

Langage calculables

- On s'intéressera à la décidabilité des langages, c'est-à-dire aux sous-ensemble de la forme

$$L \subseteq \Sigma^*$$

où Σ est un alphabet (un ensemble fini de lettres).

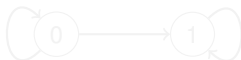
- De façon similaire, on se demandera quelles fonctions de la forme

$$f : \Sigma^* \rightarrow \Sigma^*$$

sont calculables.

- Car : la plupart des ensembles d'objets que l'on souhaite calculer se code comme un sous-ensemble de Σ^* .

Exemple : avec $\Sigma = \{0, 1, 2, \bullet\}$, le graphe



se code par le mot $2 \bullet 00 \bullet 01 \bullet 11$

- On peut même supposer que $\Sigma = \{0, 1\}^6$ est un alphabet binaire, car tout alphabet fini se code comme un sous-ensemble de Σ^* .

6. En principe, on pourrait supposer que $\Sigma = \{1\}$.

Langage calculables

- On s'intéressera à la décidabilité des langages, c'est-à-dire aux sous-ensemble de la forme

$$L \subseteq \Sigma^*$$

où Σ est un alphabet (un ensemble fini de lettres).

- De façon similaire, on se demandera quelles fonctions de la forme

$$f : \Sigma^* \rightarrow \Sigma^*$$

sont calculables.

- Car : la plupart des ensembles d'objets que l'on souhaite calculer se code comme un sous-ensemble de Σ^* .

Exemple : avec $\Sigma = \{0, 1, 2, \bullet\}$, le graphe



se code par le mot $2 \bullet 00 \bullet 01 \bullet 11$

- On peut même supposer que $\Sigma = \{0, 1\}^6$ est un alphabet binaire, car tout alphabet fini se code comme un sous-ensemble de Σ^* .

6. En principe, on pourrait supposer que $\Sigma = \{1\}$.

Langage calculables

- On s'intéressera à la décidabilité des langages, c'est-à-dire aux sous-ensemble de la forme

$$L \subseteq \Sigma^*$$

où Σ est un alphabet (un ensemble fini de lettres).

- De façon similaire, on se demandera quelles fonctions de la forme

$$f : \Sigma^* \rightarrow \Sigma^*$$

sont calculables.

- Car : la plupart des ensembles d'objets que l'on souhaite calculer se code comme un sous-ensemble de Σ^* .

Exemple : avec $\Sigma = \{0, 1, 2, \bullet\}$, le graphe



se code par le mot $2 \bullet 00 \bullet 01 \bullet 11$

- On peut même supposer que $\Sigma = \{0, 1\}^6$ est un alphabet binaire, car tout alphabet fini se code comme un sous-ensemble de Σ^* .

6. En principe, on pourrait supposer que $\Sigma = \{1\}$.

Langage calculables

- On s'intéressera à la décidabilité des langages, c'est-à-dire aux sous-ensemble de la forme

$$L \subseteq \Sigma^*$$

où Σ est un alphabet (un ensemble fini de lettres).

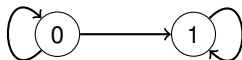
- De façon similaire, on se demandera quelles fonctions de la forme

$$f : \Sigma^* \rightarrow \Sigma^*$$

sont calculables.

- Car : la plupart des ensembles d'objets que l'on souhaite calculer se code comme un sous-ensemble de Σ^* .

Exemple : avec $\Sigma = \{0, 1, 2, \bullet\}$, le graphe



se code par le mot $2 \bullet 00 \bullet 01 \bullet 11$

- On peut même supposer que $\Sigma = \{0, 1\}^6$ est un alphabet binaire, car tout alphabet fini se code comme un sous-ensemble de Σ^* .

Langage calculables

- On s'intéressera à la décidabilité des langages, c'est-à-dire aux sous-ensemble de la forme

$$L \subseteq \Sigma^*$$

où Σ est un alphabet (un ensemble fini de lettres).

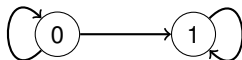
- De façon similaire, on se demandera quelles fonctions de la forme

$$f : \Sigma^* \rightarrow \Sigma^*$$

sont calculables.

- Car : la plupart des ensembles d'objets que l'on souhaite calculer se code comme un sous-ensemble de Σ^* .

Exemple : avec $\Sigma = \{0, 1, 2, \bullet\}$, le graphe



se code par le mot $2 \bullet 00 \bullet 01 \bullet 11$

- On peut même supposer que $\Sigma = \{0, 1\}^6$ est un alphabet binaire, car tout alphabet fini se code comme un sous-ensemble de Σ^* .

6. En principe, on pourrait supposer que $\Sigma = \{1\}$.